

An Exploration of Sentiment Analysis Methods to Study Student Perspectives on Artificial Intelligence

Professor Elodie Fourquet, Anika Gurjar

Colgate University, Department of Computer Science

Preface

Prior to following this quarto document, it may be helpful to read and complete a quick refresher on R basics. I recommend the online book:

R for Data Science (2e) by Hadley Wickham, Mine Çetinkaya-Rundel, and Garrett Grolemund (<https://r4ds.hadley.nz/>)

This book provides helpful information on what packages to install in R and other techniques in R. I recommend going through the first section of the book titled “Whole Game.” I have also included a more detailed overview of helpful chapters from the book if you want to briefly skim through them:

Chapter 1 - Data Visualization

- Creating and customizing plots

Chapter 3 - Data Transformation

- Filtering, selecting, counting, piping(%>%)

Chapter 5 - Data Tidying

- Tidy data, library(tidyverse)

Chapter 7 - Data Import

- Reading text files

Chapter 10 - Exploratory Data Analysis

- Summarizing data and basic plotting with ggplot2

Chapter 14 - Strings

- String manipulation with stringr - important for filtering tokens

Additionally, a majority of the information that I am sharing in this quarto document is information that I learned from **Chapter 2 - “Sentiment analysis with tidy data”** of the online book:

Text Mining with R: A Tidy Approach by Julia Silge and David Robinson (<https://www.tidytextmining.com/sentiment>)

Throughout the quarto document, when possible, I will cite the sub-section within chapter 2 of this book that I got certain information from. For example, 2.1, 2.2, etc.

A lot of the information that I am sharing in this quarto document is also information that I learned from **Chapter 6 - "From Text to Numbers"** of the book:

Mapping Texts: Computational Text Analysis for the Social Sciences by Dustin S. Stoltz and Marshall A. Taylor

I will also cite this book throughout this quarto document to show where I got my information from.

Finding the top word frequencies of a text file:

(I learned how to do this from chapter 6 "From Text to Numbers" of *Mapping Texts: Computational Text Analysis for the Social Sciences* by Dustin S. Stoltz and Marshall A. Taylor).

First, we have to open the packages that we need.

```
library(tidytext) #provides function to break text into chunks called tokens (can be chunked)
library(dplyr) #provides functionality in data manipulation, such as filtering text
```

Attaching package: 'dplyr'

The following objects are masked from 'package:stats':

filter, lag

The following objects are masked from 'package:base':

intersect, setdiff, setequal, union

```
library(stringr) #provides functionality that simplifies string manipulation, extraction
library(ggplot2) #provides functionality for data visualization such as making graphs, plotting
library(tidyr) #provides further functionality to be able to tidy datasets by creating separate
```

Next, we open the text file (student1_cleaned.txt) from our downloads into R. We tell R to read the lines of the text file, and give the text file the name: text_data.

```
text_data <- readLines("/Users/anikagurjar/downloads/student1_cleaned.txt")
```

Next, we create a type of data frame called a tibble from text_data. In the tibble, each row shows one line from text_data. We name this tibble: text_df.

```
text_df <- tibble(line = 1:length(text_data), text = text_data)
```

Next, we load `stop_words`, which is a built-in data set from the `tidytext` package. It contains a tibble of words such as "and", "the", and "of". These words do not have much meaning on their own.

```
data("stop_words")
```

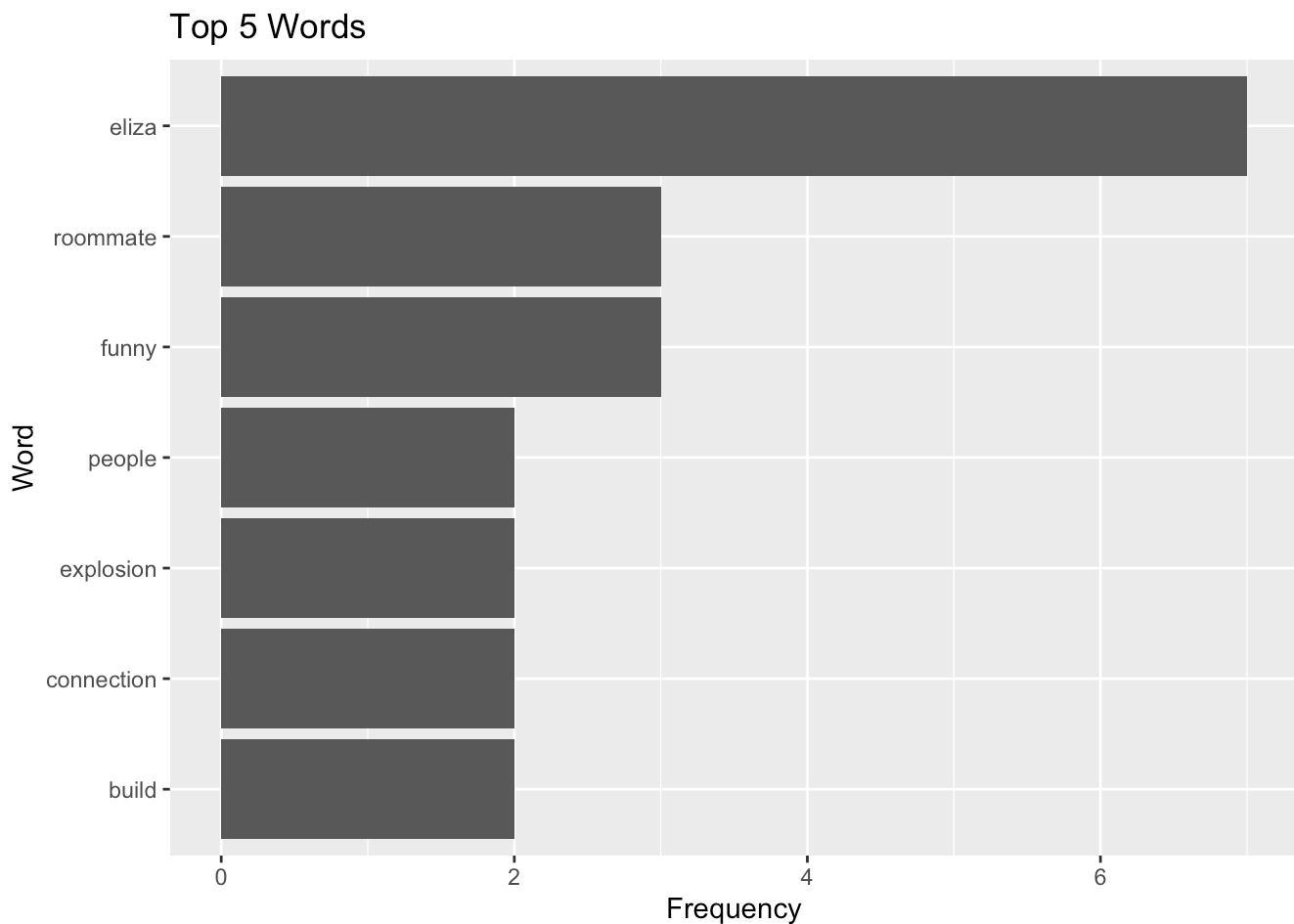
Next, we begin to clean our text file in order to prepare it for text analysis. The symbols "`%>%`" used below is called piping, and it is used to chain together multiple operations.

```
clean_df <- text_df %>%  
  unnest_tokens(word, text) %>% #tokenize(split into words)  
  filter(!word %in% stop_words$word) %>% #remove stop words  
  filter(!str_detect(word, "\\d")) %>% #remove numbers  
  filter(str_length(word) > 4) #remove words less than 4 letters
```

Lastly, we find the top 5 most frequently occurring words in the cleaned text and create a data visualization to representing them. In the case that there is a tie in the frequency of words, the data visualization may display more than 5 words.

```
clean_df %>%  
  count(word, sort = TRUE) %>% #counts how often each word occurs and then sorts the word  
  top_n(5) %>% #selects top 5 most frequent words  
  ggplot(aes(x = reorder(word, n), y = n)) + #creates a chart, assigning x to the word re  
  geom_col() + #draws bar plots  
  coord_flip() + #flips x and y axes to make plot horizontal  
  labs(title = "Top 5 Words", x = "Word", y = "Frequency") #add labels to plot
```

Selecting by n



While the above plot gives us a good understanding of the most frequent words, it does not show the larger themes of students thoughts about AI. This is because the top words are specific to student 1's text file. So, I made a larger text file in which I combined 12 student texts. I named this file "combined_data.txt"

I then ran the same code on combined_data.txt and produced the following plot:

```
library(tidytext)
library(dplyr)
library(stringr)
library(ggplot2)
library(tidyr)

text_data <- readLines("/Users/anikagurjar/downloads/combined_data.txt")

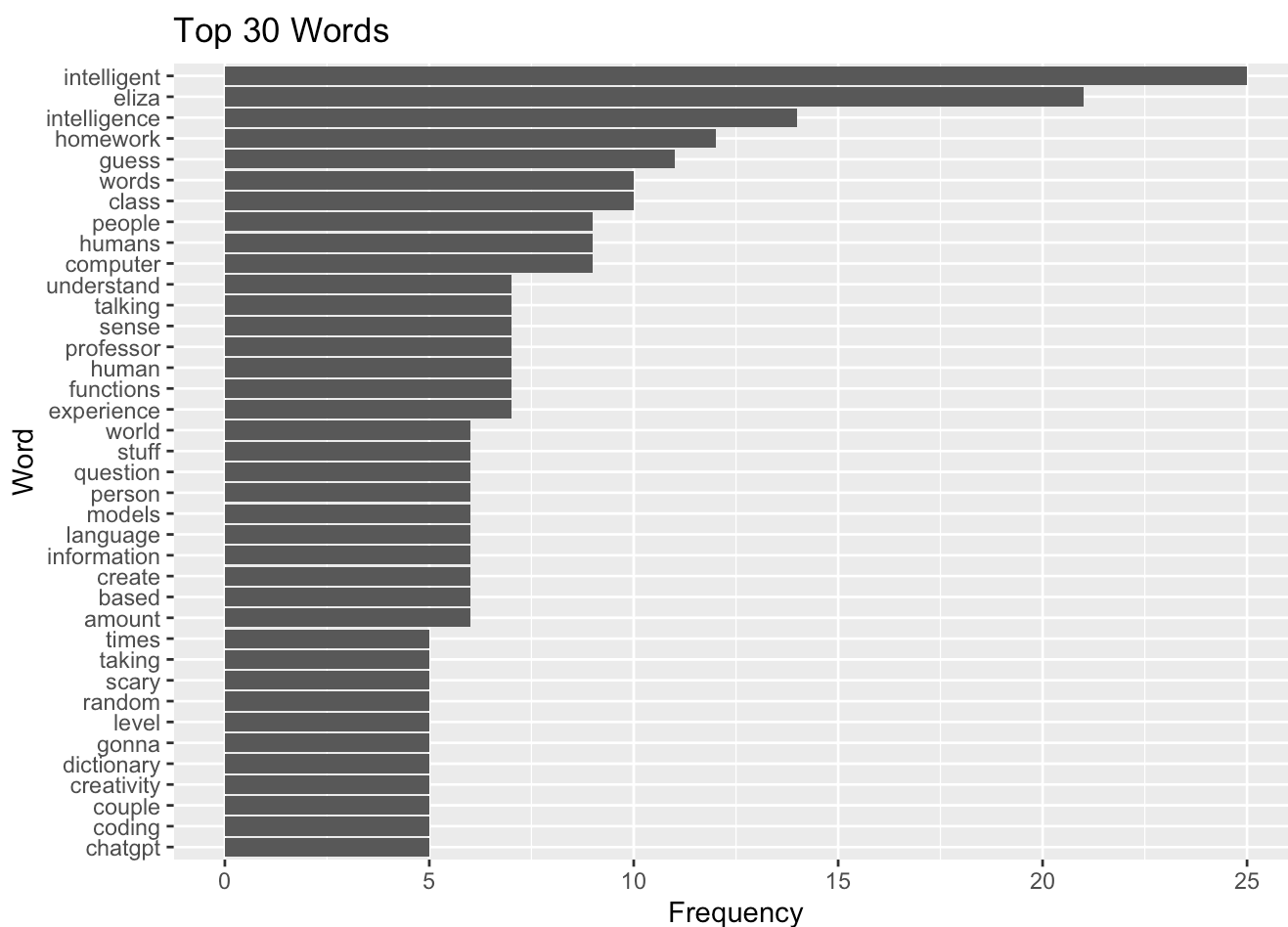
text_df <- tibble(line = 1:length(text_data), text = text_data)

data("stop_words")

clean_text <- text_df %>%
  unnest_tokens(word, text) %>%
  filter(!word %in% stop_words$word) %>%
  filter(!str_detect(word, "\\d")) %>%
  filter(str_length(word) > 4)
```

```
clean_text %>%
  count(word, sort = TRUE) %>%
  top_n(30) %>%
  ggplot(aes(x = reorder(word, n), y = n)) +
  geom_col() +
  coord_flip() +
  labs(title = "Top 30 Words", x = "Word", y = "Frequency")
```

Selecting by n



The above plot can give us a better understanding of the main words and themes that students are mentioning when discussing their thoughts on AI.

Conducting sentiment analysis of a text file:

(Chapters used from [Text Mining with R: A Tidy Approach](#), by Julia Silge and David Robinson: 2.1, 2.2, 2.3, and 2.4).

Now we can continue our analysis of our text by beginning to conduct sentiment analysis on the text file, in order to derive meaning from the text.

I will first explain the step by step process of how to complete this sentiment analysis: First, we must open the text file in R and clean it. Then, we load the bing sentiment lexicon which is included in the

tidytext package. The bing sentiment lexicon is a list of words labeled as positive or negative. Next, we create a variable called "bing_words", which only contains words that appear in both the text being analyzed and the bing lexicon. Then, we count the number of times that positive and negative sentiments occur and sort them separately. Lastly, we create a bar chart that shows the total number of positive words and the total number of negative words in the text.

The complete code to go along with these steps is shown below:

```
library(tidytext)
library(dplyr)
library(stringr)
library(ggplot2)
library(tidyr)

text_data <- readLines("/Users/anikagurjar/downloads/student1_cleaned.txt")

text_df <- tibble(line = 1:length(text_data), text = text_data)

data("stop_words")

clean_text <- text_df %>%
  unnest_tokens(word, text) %>%
  filter(!word %in% stop_words$word) %>%
  filter(!str_detect(word, "\\d")) %>%
  filter(str_length(word) > 4)

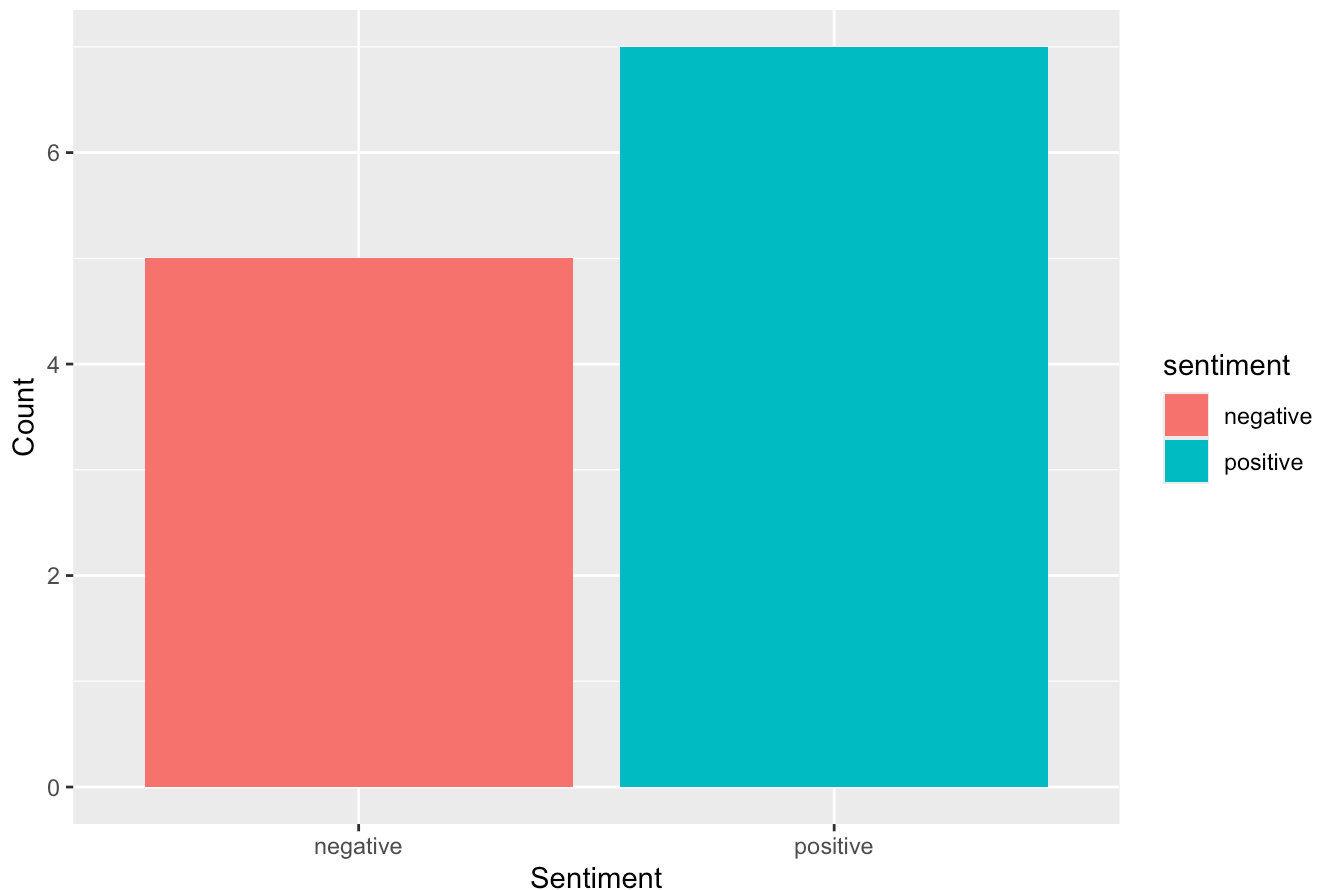
bing <- get_sentiments("bing") #loads bing sentiment lexicon, which is a list of words l

bing_words <- clean_text %>% #only keeps words from cleaned text that also appear in bi
  inner_join(bing, by = "word")

bing_counts <- bing_words %>% # counts the number of times positive and negative sentim
  count(sentiment, sort = TRUE)

ggplot(bing_counts, aes(x = sentiment, y = n, fill = sentiment)) + # creates a b
  geom_col(show.legend = TRUE) +
  labs(title = "Positive vs. Negative Words in Text (Sentiment Count Using Bing Lexicon)"
       x = "Sentiment", y = "Count")
```

Positive vs. Negative Words in Text (Sentiment Count Using Bing Lexicon)



The above plot shows the amount of positive vs. negative words in student 1's text. It is best to do this type of sentiment analysis using the Bing lexicon with smaller text files. This is because, if you use larger text the amount of positive vs. negative words tend to even out over time which does not help us to do any sentiment analysis. Because of this, it would not be useful to conduct the above text analysis on my large text file, "combined_data.txt"

However I will conduct the sentiment analysis using the Bing lexicon a few more times to other student texts.

Student 2:

```
library(tidytext)
library(dplyr)
library(stringr)
library(ggplot2)
library(tidyr)

text_data <- readLines("/Users/anikagurjar/downloads/student2_cleaned.txt")
```

```
Warning in readLines("/Users/anikagurjar/downloads/student2_cleaned.txt"):
incomplete final line found on
'/Users/anikagurjar/downloads/student2_cleaned.txt'
```

```

text_df <- tibble(line = 1:length(text_data), text = text_data)

data("stop_words")

clean_text <- text_df %>%
  unnest_tokens(word, text) %>%
  filter(!word %in% stop_words$word) %>%
  filter(!str_detect(word, "\\d")) %>%
  filter(str_length(word) > 4)

bing <- get_sentiments("bing") #loads bing sentiment lexicon, which is a list of words l

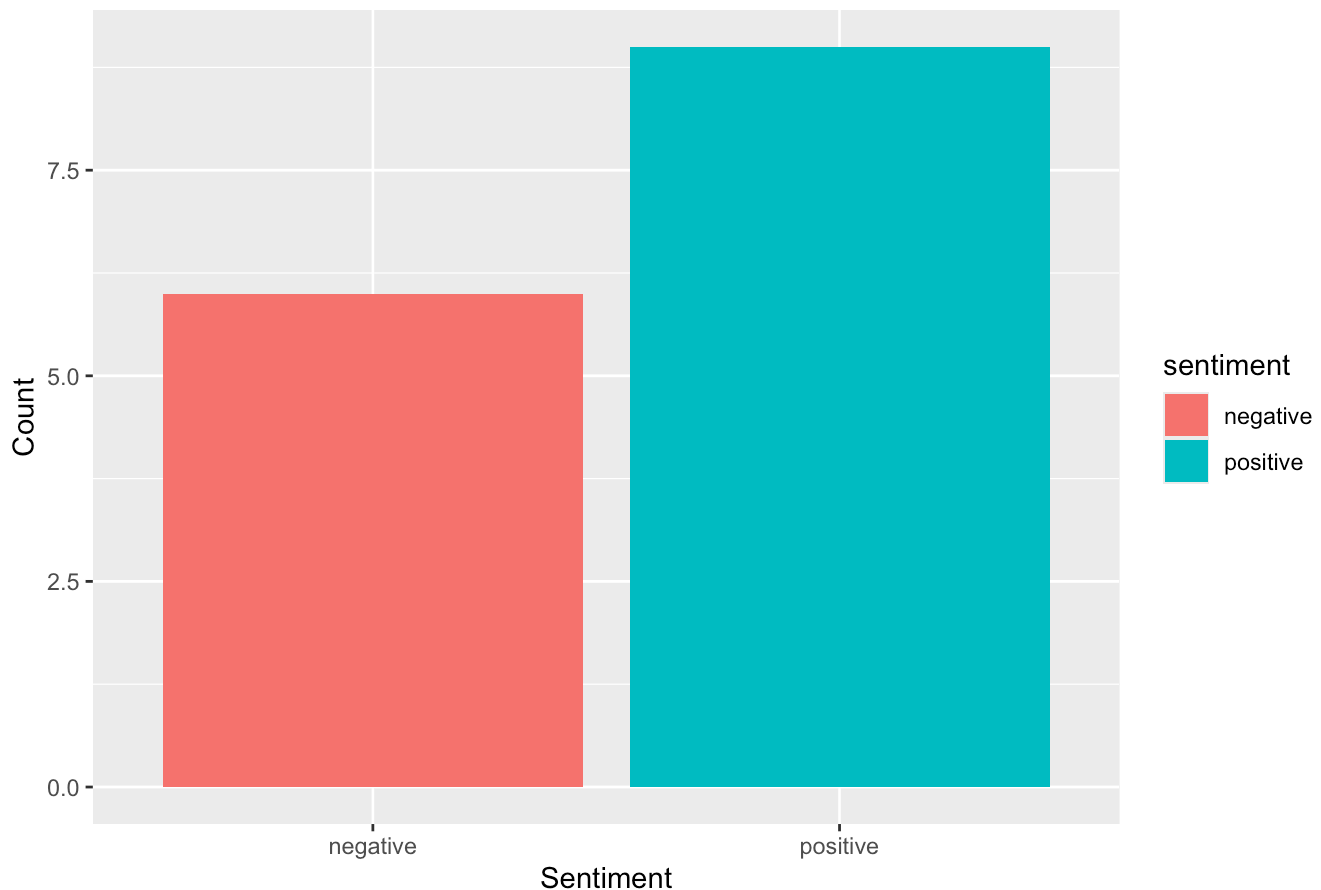
bing_words <- clean_text %>% #only keeps words from cleaned text that also appear in bi
  inner_join(bing, by = "word")

bing_counts <- bing_words %>% # counts the number of times positive and negative sentim
  count(sentiment, sort = TRUE)

ggplot(bing_counts, aes(x = sentiment, y = n, fill = sentiment)) + # creates a b
  geom_col(show.legend = TRUE) +
  labs(title = "Positive vs. Negative Words in Text (Sentiment Count Using Bing Lexicon)"
       x = "Sentiment", y = "Count")

```

Positive vs. Negative Words in Text (Sentiment Count Using Bing Lexicon)




```
library(tidytext)
library(dplyr)
library(stringr)
library(ggplot2)
library(tidyr)

text_data <- readLines("/Users/anikagurjar/downloads/student10_cleaned.txt")
```

Warning in readLines("/Users/anikagurjar/downloads/student10_cleaned.txt"):
incomplete final line found on
'/Users/anikagurjar/downloads/student10_cleaned.txt'

```
text_df <- tibble(line = 1:length(text_data), text = text_data)

data("stop_words")

clean_text <- text_df %>%
  unnest_tokens(word, text) %>%
  filter(!word %in% stop_words$word) %>%
  filter(!str_detect(word, "\\d")) %>%
  filter(str_length(word) > 4)

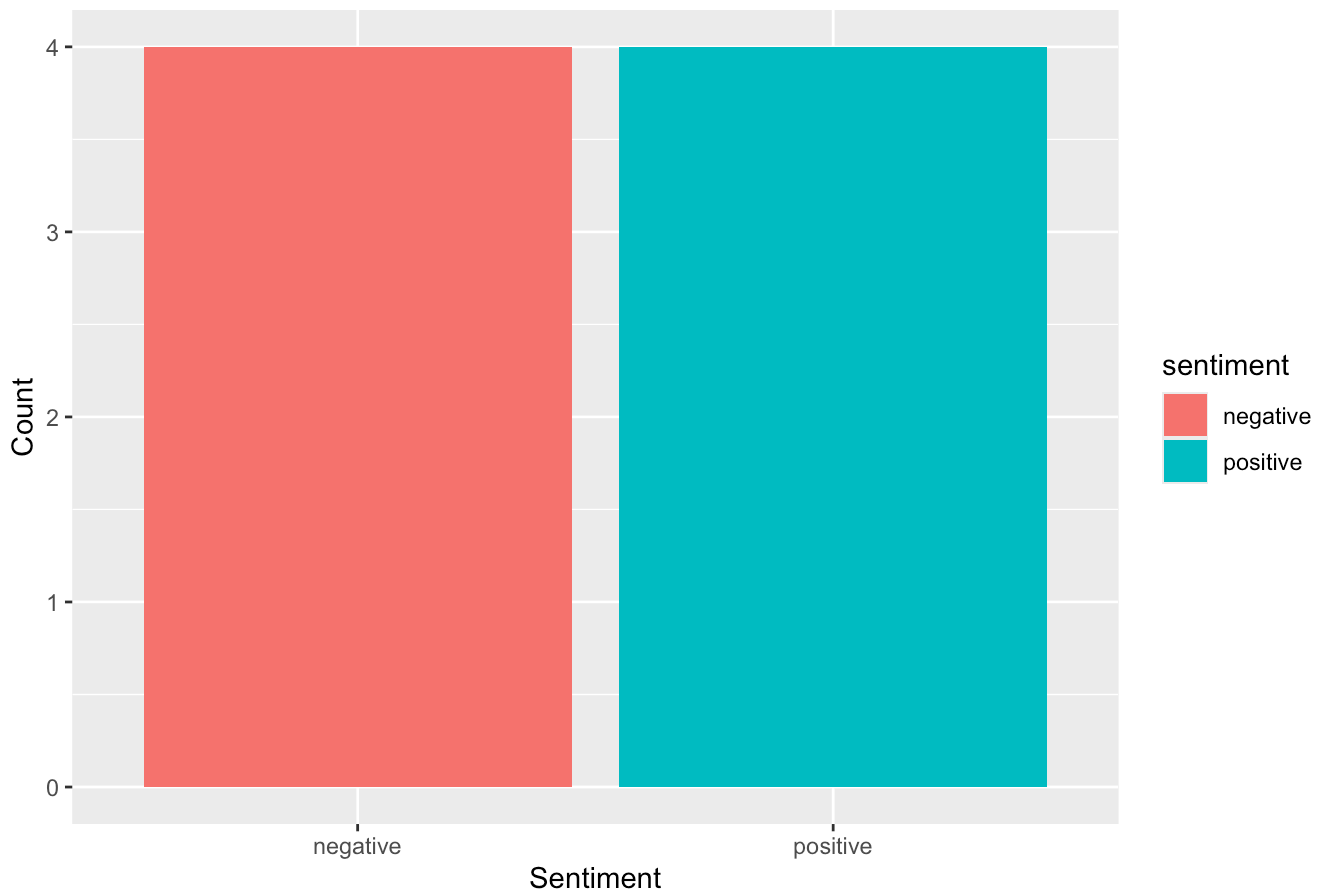
bing <- get_sentiments("bing") #loads bing sentiment lexicon, which is a list of words l

bing_words <- clean_text %>% #only keeps words from cleaned text that also appear in bi
  inner_join(bing, by = "word")

bing_counts <- bing_words %>% # counts the number of times positive and negative sentim
  count(sentiment, sort = TRUE)

ggplot(bing_counts, aes(x = sentiment, y = n, fill = sentiment)) + # creates a b
  geom_col(show.legend = TRUE) +
  labs(title = "Positive vs. Negative Words in Text (Sentiment Count Using Bing Lexicon)"
       x = "Sentiment", y = "Count")
```

Positive vs. Negative Words in Text (Sentiment Count Using Bing Lexicon)



Student 11:

```
library(tidytext)
library(dplyr)
library(stringr)
library(ggplot2)
library(tidyr)

text_data <- readLines("/Users/anikagurjar/downloads/student11_cleaned.txt")
```

Warning in readLines("/Users/anikagurjar/downloads/student11_cleaned.txt"):
incomplete final line found on
'/Users/anikagurjar/downloads/student11_cleaned.txt'

```
text_df <- tibble(line = 1:length(text_data), text = text_data)

data("stop_words")

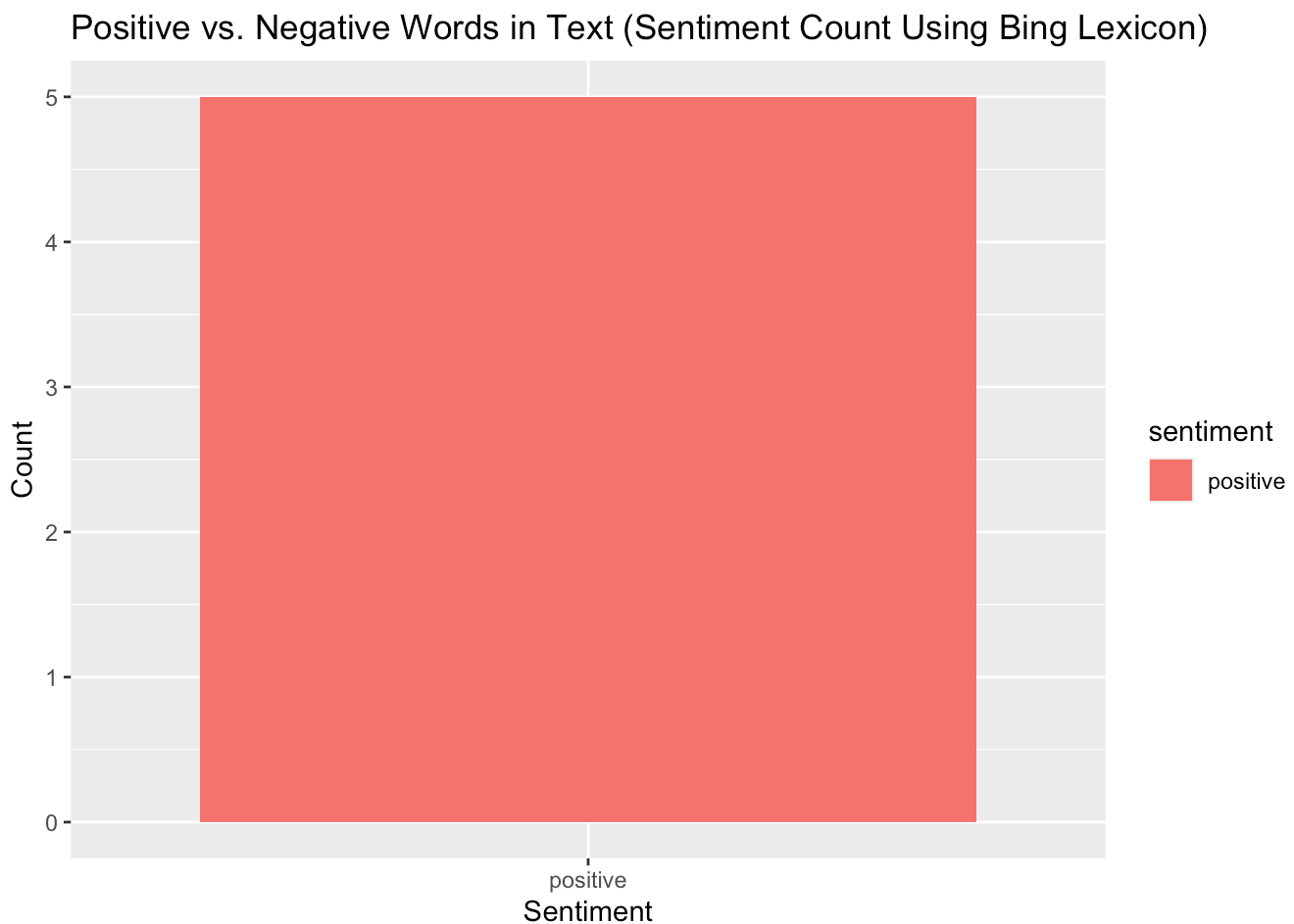
clean_text <- text_df %>%
  unnest_tokens(word, text) %>%
  filter(!word %in% stop_words$word) %>%
  filter(!str_detect(word, "\\d")) %>%
  filter(str_length(word) > 4)
```

```
bing <- get_sentiments("bing") #loads bing sentiment lexicon, which is a list of words l

bing_words <- clean_text %>% #only keeps words from cleaned text that also appear in bi
  inner_join(bing, by = "word")

bing_counts <- bing_words %>% # counts the number of times positive and negative sentim
  count(sentiment, sort = TRUE)

ggplot(bing_counts, aes(x = sentiment, y = n, fill = sentiment)) + # creates a b
  geom_col(show.legend = TRUE) +
  labs(title = "Positive vs. Negative Words in Text (Sentiment Count Using Bing Lexicon)"
       x = "Sentiment", y = "Count")
```



The different sentiment plots that we have seen above suggest that perhaps different students have different perspectives on if AI is positive or negative.

It is important to note that we should not fully trust the bing lexicon in classifying words as postive or negative. For example, the bing lexicon classifies “funny” as a negative word; as in a funny smell, or a funny feeling. It is possible that a student may have used funny as a positive word but the bing lexicon categorized it as a negative word.

If we want, we can also see which words from the text file contributed most to the postive and negative counts. We can do so using the following code:

```

library(tidytext)
library(dplyr)
library(stringr)
library(ggplot2)
library(tidyr)

text_data <- readLines("/Users/anikagurjar/downloads/student1_cleaned.txt")

text_df <- tibble(line = 1:length(text_data), text = text_data)

data("stop_words")

clean_text <- text_df %>%
  unnest_tokens(word, text) %>%
  filter(!word %in% stop_words$word) %>%
  filter(!str_detect(word, "\\d")) %>%
  filter(str_length(word) > 4)

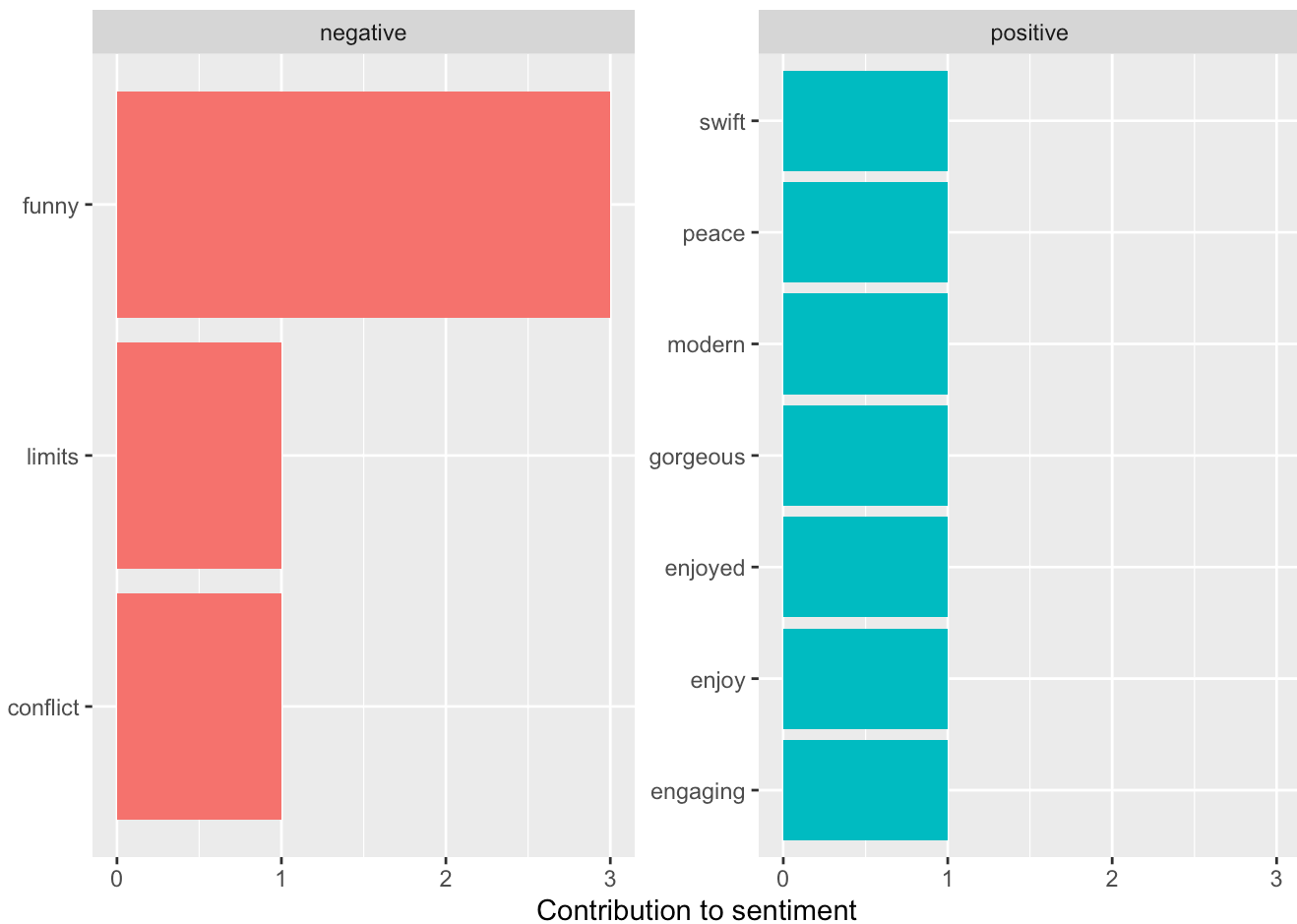
bing <- get_sentiments("bing")

bing_words <- clean_text %>%
  inner_join(bing, by = "word")

bing_counts <- bing_words %>%
  count(sentiment, word, sort = TRUE) %>%
  ungroup()

bing_counts %>%
  group_by(sentiment) %>%
  slice_max(n, n = 10) %>%
  ungroup() %>%
  mutate(word = reorder(word, n)) %>%
  ggplot(aes(n, word, fill = sentiment)) +
  geom_col(show.legend = FALSE) +
  facet_wrap(~sentiment, scales = "free_y") +
  labs(x = "Contribution to sentiment",
       y = NULL)

```



The above plot shows us that the word “funny” had the highest contribution to the negative word count in student 1’s text file. However I know that in student 1’s text the word “funny” was used in a positive way. In order to fix this misunderstanding I can add the word “funny” to custom stop words and filter it out of the text during the cleaning process.

The following code is edited to filter out the word “funny” as a stop word:

```
library(tidytext)
library(dplyr)
library(stringr)
library(ggplot2)
library(tidyr)

text_data <- readLines("/Users/anikagurjar/downloads/student1_cleaned.txt")

text_df <- tibble(line = 1:length(text_data), text = text_data)

data("stop_words")

custom_stop_words <- bind_rows(tibble(word = c("funny"),
                                       lexicon = c("custom")),
                              stop_words)

#we add the word "funny" to the custom stop words
```

```

clean_text <- text_df %>%
  unnest_tokens(word, text) %>%
  filter(!word %in% custom_stop_words$word) %>% #we filter out the custom stop words, whi
  filter(!str_detect(word, "\\d")) %>%
  filter(str_length(word) > 4)

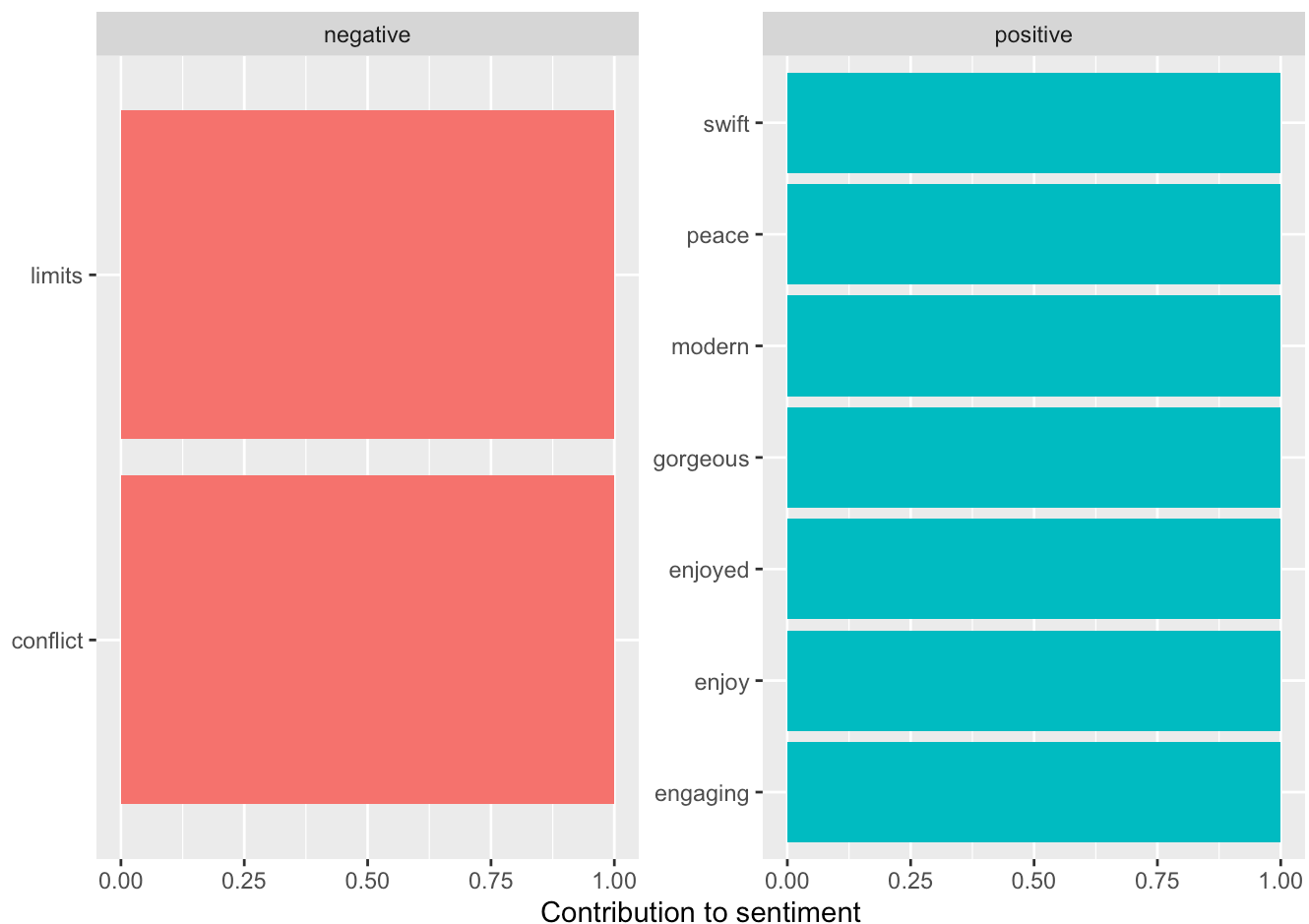
bing <- get_sentiments("bing")

bing_words <- clean_text %>%
  inner_join(bing, by = "word")

bing_counts <- bing_words %>%
  count(sentiment, word, sort = TRUE) %>%
  ungroup()

bing_counts %>%
  group_by(sentiment) %>%
  slice_max(n, n = 10) %>%
  ungroup() %>%
  mutate(word = reorder(word, n)) %>%
  ggplot(aes(n, word, fill = sentiment)) +
  geom_col(show.legend = FALSE) +
  facet_wrap(~sentiment, scales = "free_y") +
  labs(x = "Contribution to sentiment",
       y = NULL)

```



Now, we can see that the plot does not show the word “funny” anymore. Now that we have filtered out the word “funny”, we can go back and remake the bar chart of positive vs. negative words in the text:

```
library(tidytext)
library(dplyr)
library(stringr)
library(ggplot2)
library(tidyr)

text_data <- readLines("/Users/anikagurjar/downloads/student1_cleaned.txt")

text_df <- tibble(line = 1:length(text_data), text = text_data)

data("stop_words")

custom_stop_words <- bind_rows(tibble(word = c("funny"),
                                       lexicon = c("custom")),
                              stop_words)

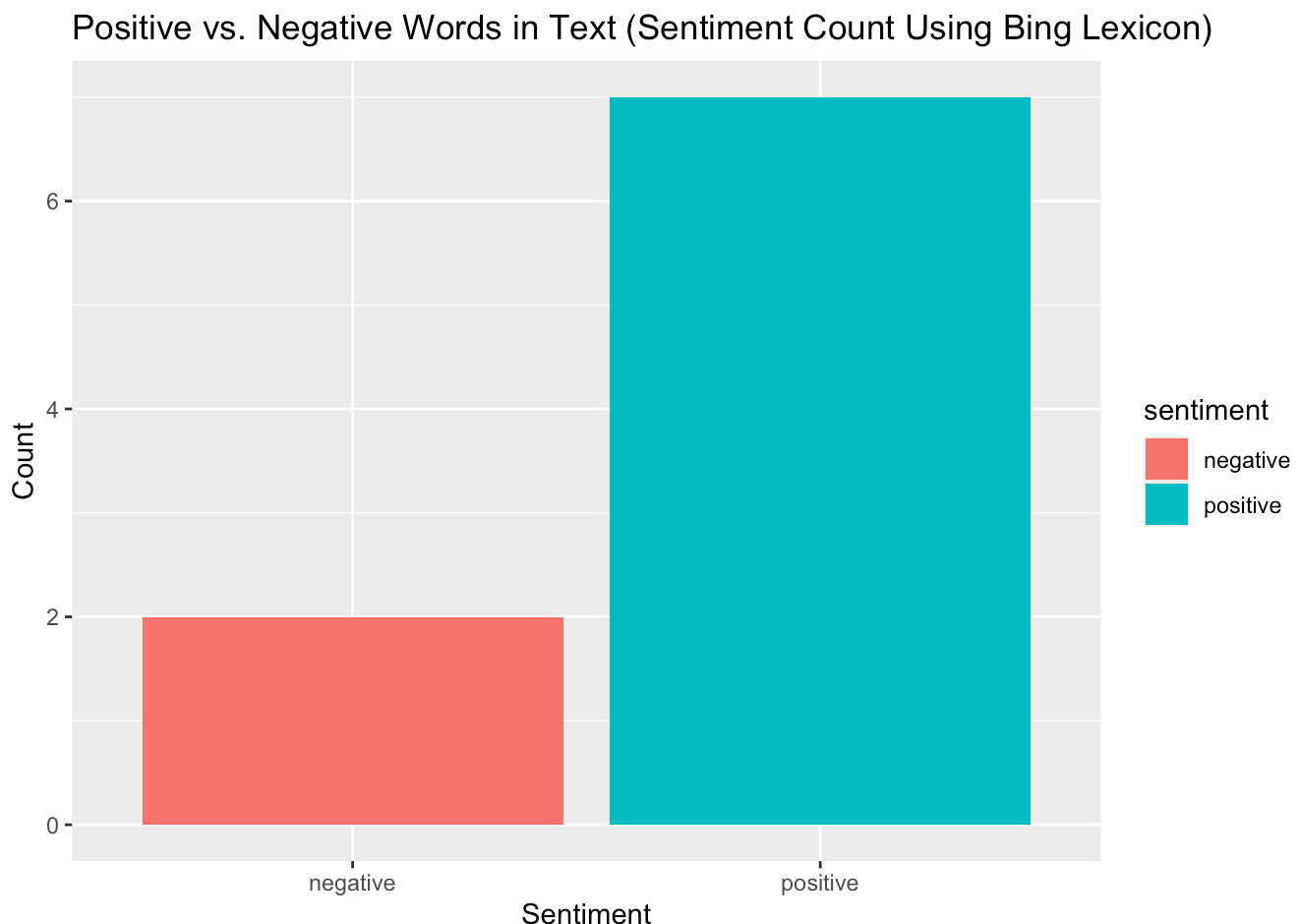
clean_text <- text_df %>%
  unnest_tokens(word, text) %>%
  filter(!word %in% custom_stop_words$word) %>%
  filter(!str_detect(word, "\\d")) %>%
  filter(str_length(word) > 4)
```

```
bing_lexicon <- get_sentiments("bing")

bing_words <- clean_text %>%
  inner_join(bing, by = "word")

bing_counts <- bing_words %>%
  count(sentiment, sort = TRUE)

ggplot(bing_counts, aes(x = sentiment, y = n, fill = sentiment)) +
  geom_col(show.legend = TRUE) +
  labs(title = "Positive vs. Negative Words in Text (Sentiment Count Using Bing Lexicon)",
       x = "Sentiment", y = "Count")
```



This above plot that analyzes student 1's text now displays a significantly less count of negative words than it did before filtering out the word "funny."

Another way that we can conduct sentiment analysis on a text is by finding how sentiment changes over time through the text. We can do this by breaking the text into chunks, and then analyzing the positive vs. negative sentiment in each chunk to calculate the net sentiment of each chunk. Then we can create a visualization that shows the change in net sentiment over time. The code to do this is included below, with comments in the code explaining how the process is done:


```

library(tidytext)
library(dplyr)
library(stringr)
library(ggplot2)
library(tidyr)

text_data <- readLines("/Users/anikagurjar/downloads/student1_cleaned.txt")

text_df <- tibble(line = 1:length(text_data), text = text_data)

data("stop_words")

custom_stop_words <- bind_rows(tibble(word = c("funny"),
                                       lexicon = c("custom")),
                                stop_words)

clean_text <- text_df %>%
  unnest_tokens(word, text) %>%
  filter(!word %in% custom_stop_words$word) %>%
  filter(!str_detect(word, "\\d")) %>%
  filter(str_length(word) > 4)

bing <- get_sentiments("bing")

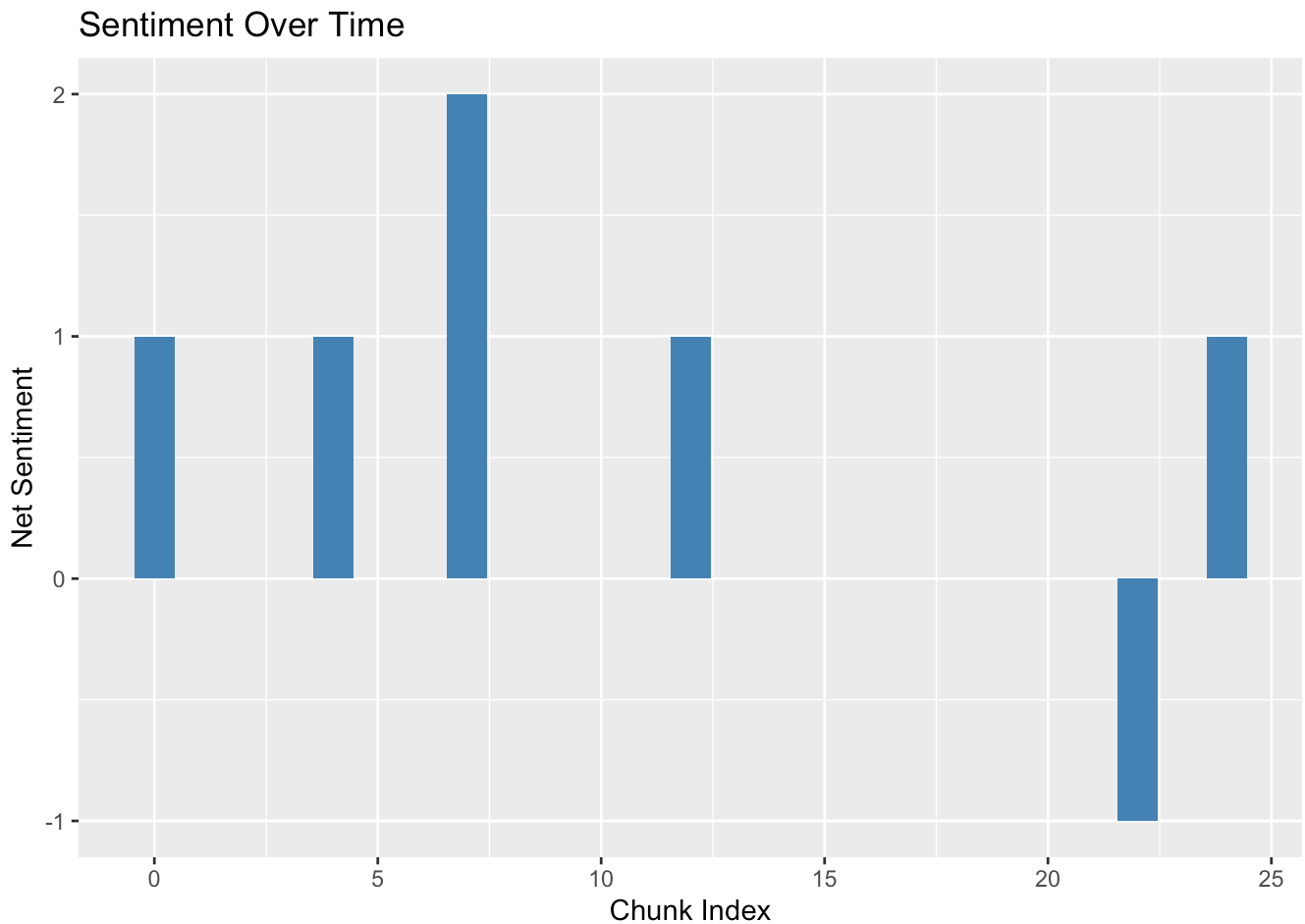
bing_words <- clean_text %>%
  inner_join(bing, by = "word")

bing_counts <- bing_words %>%
  count(sentiment, sort = TRUE)

sentiment_trajectory <- clean_text %>%
  inner_join(bing, by = "word") %>% #add sentiment label to each word
  mutate(index = line %/% 2) %>% #divides text into chunks, every two lines
  count(index, sentiment) %>% #counts the number of positive and negative words in each
  pivot_wider(names_from = sentiment, values_from = n, values_fill = 0) %>% #makes one
  mutate(net_sentiment = positive - negative) #calculates net sentiment per chunk

ggplot(sentiment_trajectory, aes(x = index, y = net_sentiment)) +
  geom_col(fill = "steelblue") +
  labs(title = "Sentiment Over Time",
       x = "Chunk Index", y = "Net Sentiment")

```



Data visualization as word clouds:

(Chapters used from [Text Mining with R: A Tidy Approach](#), by Julia Silge and David Robinson: 2.5).

We can create visualizations of the most frequently occurring words in a text using the wordcloud package: (In order to do this, you will have to install the wordcloud package by running the following in R: `install.packages("wordcloud")`)

```
library(tidytext)
library(dplyr)
library(stringr)
library(ggplot2)
library(tidyr)

text_data <- readLines("/Users/anikagurjar/downloads/combined_data.txt")

text_df <- tibble(line = 1:length(text_data), text = text_data)

data("stop_words")

clean_text <- text_df %>%
  unnest_tokens(word, text) %>%
  filter(!word %in% stop_words$word) %>%
```

```
library(wordcloud)
```

```
clean_text %>%
  anti_join(stop_words) %>%
  count(word) %>%
  with(wordcloud(word, n, max.words = 100))
```

A word cloud visualization of terms related to AI. The words are arranged in a dense, overlapping manner. The most prominent words, shown in the largest font, are 'intelligent' and 'homework'. Other large words include 'computer', 'words', 'humans', 'class', 'functions', 'amount', 'models', 'sense', 'intelligence', 'language', 'chatgpt', 'people's', 'world', 'couple', 'creativity', 'advanced', 'level', 'understand', 'guess', 'eliza', 'information', 'artificial', 'times', 'create', 'super', 'coding', 'similar', 'dictionary', 'helped', 'upset', 'shakespeare', 'shakespearean', 'gonna', 'funny', 'system', 'roommate', 'connection', 'generate', 'learning', 'emotion', 'story', 'helpful', 'started', 'stuff', 'emotional', 'completely', 'learn', 'future', 'talked', 'learned', 'built', 'chat', 'gpt', 'output', 'input', 'science', 'selected', 'feelings', 'taking', 'complex', 'knowledge', 'eventually', 'scary', 'emotions', 'feeling', 'question', 'talking', 'person', 'random', 'incorrect', 'people', 'patterns', 'based', 'professor', 'video', 'forward', 'basic', 'makes', 'coming', 'sources', 'pretty', 'patterns', 'based', 'professor', 'video', 'forward', 'basic', 'makes', 'coming', 'sources', 'pretty'. The words are in various shades of gray and black, and are oriented in different directions, creating a dynamic and abstract visual representation of the data.

```
library(tidytext)
library(dplyr)
library(stringr)
library(ggplot2)
```

```

library(tidyr)

text_data <- readLines("/Users/anikagurjar/downloads/combined_data.txt")

text_df <- tibble(line = 1:length(text_data), text = text_data)

data("stop_words")

clean_text <- text_df %>%
  unnest_tokens(word, text) %>%
  filter(!word %in% stop_words$word) %>%
  filter(!str_detect(word, "\\d")) %>%
  filter(str_length(word) > 4)

library(wordcloud)
library(tibble)

word_counts <- clean_text %>%
  inner_join(get_sentiments("bing")) %>%
  count(word, sentiment, sort = TRUE) %>%
  pivot_wider(names_from = sentiment, values_from = n, values_fill = 0)

```

Joining with `by = join\_by(word)`

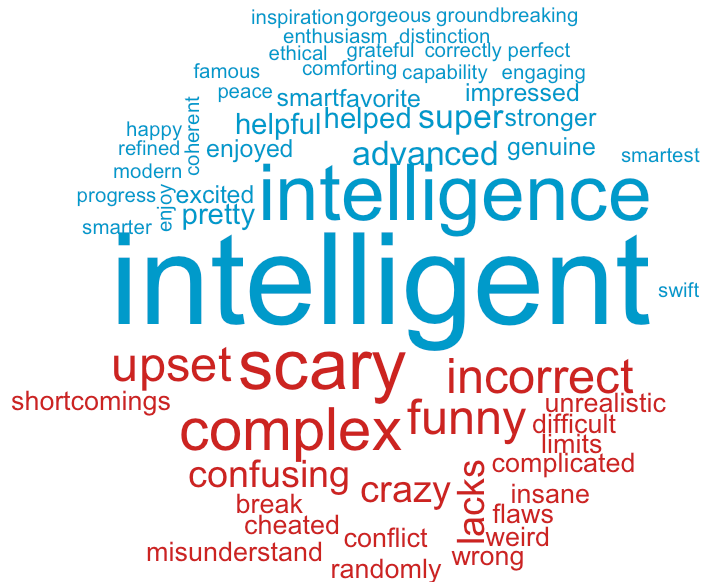
```

# Convert to matrix for comparison.cloud
word_matrix <- as.matrix(column_to_rownames(word_counts, "word"))

comparison.cloud(word_matrix,
  colors = c("deepskyblue3", "firebrick3"),
  max.words = 100)

```

positive



negative

Other sentiment lexicons that can be used:

(Chapters used from Text Mining with R: A Tidy Approach, by Julia Silge and David Robinson: 2.1).

So far, I have only discussed the Bing sentiment lexicon that contains a list of words categorized as either positive or negative.

```
library(tidytext)

get_sentiments("bing")
```

```
# A tibble: 6,786 × 2
  word      sentiment
<chr>    <chr>
1 2-faces    negative
2 abnormal  negative
3 abolish    negative
4 abominable negative
5 abominably negative
6 abominate  negative
7 abomination negative
8 abort      negative
9 aborted    negative
```

```
10 abortions      negative
# i 6,776 more rows
```

However there are also other sentiment lexicons that can be used!

There is the AFINN lexicon. This lexicon assigns words with a numerical score from -5 to 5, with a negative score indicating a more negative sentiment and a positive score indicating a more positive sentiment. You can download the AFINN lexicon and then enter the following into R to view it:

```
get_sentiments("afinn")
```

There is also the nrc lexicon. This lexicon categorizes words binarily (yes or no) into categories of different sentiments such as positive, negative, anger, anticipation, disgust, fear, joy, sadness, surprise, and trust. You can download the nrc lexicon and then enter the following into R to view it:

```
get_sentiments("nrc")
```